



UNIVERSITÀ DEGLI STUDI DI SALERNO

INTELLIGENZA ARTIFICIALE: METODI ED APPLICAZIONI

TORCS: programmiamo il nostro primo Agente

ESEGUIRE IL CLIENT

Avviare Client Torcs Python

Configurare un ambiente **Python 3.11.x**

```
conda create -n torcs-env python=3.11
```

```
conda activate torcs-env
```

```
conda install -n torcs-env numpy scipy scikit-learn matplotlib pandas
```

```
conda install -n torcs-env -c conda-forge gymnasium
```

```
pip install torch tensorflow keras
```

Avviare TORCS

Avvia il Server Torcs: Fai doppio clic su `\torcs\wtorcs.exe`

In TORCS:

Vai su Race → Practice → Configure Race

Seleziona una pista

Imposta scr_server come driver

Conferma con Accept

Avvia la sessione di pratica:

Vai su Race → Practice → New Race

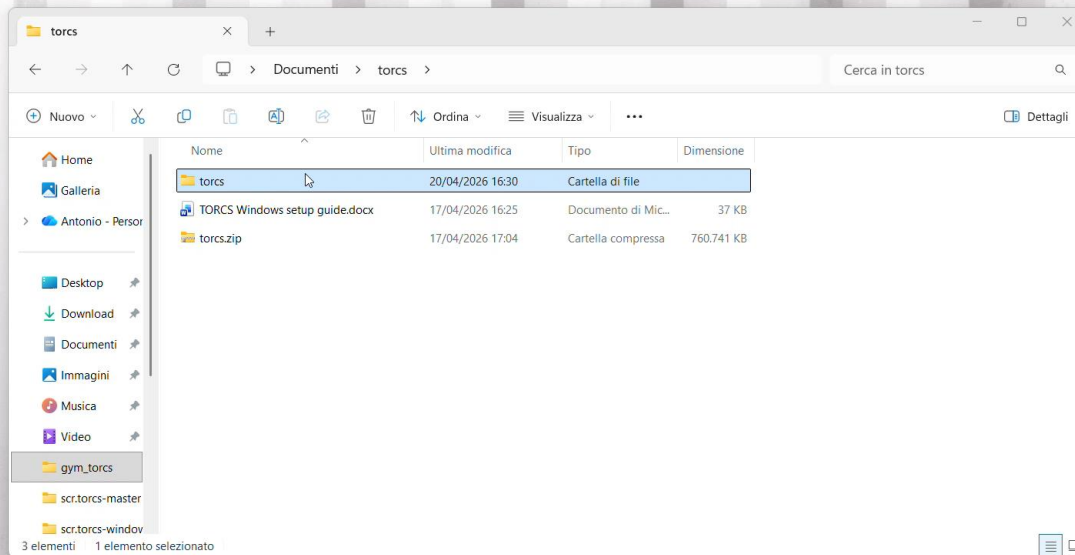
TORCS mostrerà una schermata blu di attesa

Apri PowerShell, attiva l'ambiente virtuale ed esegui il client:



```
(base) C:\Users\anpar>conda activate torcs-env  
(torcs-env) C:\Users\anpar>cd Documents\torcs\torcs\myrace\gym_torcs  
(torcs-env) C:\Users\anpar\Documents\torcs\torcs\myrace\gym_torcs>python torcs_jm_par.py|
```

Esecuzione di TORCS con client di esempio



OPENING THE BOX

API TORCS / INPUT DEL NOSTRO ALGORITMO

Spazio degli stati principali

Nome Sensore	Range	Descrizione
angle	$[-\pi, +\pi](\text{rad})$	Angolo tra l'asse longitudinale (direzione) dell'auto e l'asse tangente al tracciato.
curLapTime	$[0, +\infty) (s)$	Tempo trascorso durante il giro corrente.
damage	$[0, +\infty]$	Danno attuale dell'auto (più alto è il valore, più alto è il danno).
distFromStart	$[0, +\infty) (m)$	Distanza della vettura dalla linea di partenza lungo la linea del tracciato
distRaced	$[0, +\infty) (m)$	Distanza percorsa dall'auto dall'inizio della gara

API TORCS / INPUT DEL NOSTRO ALGORITMO

Spazio degli stati principali

Nome Sensore	Range	Descrizione
fuel	$[0, +\infty)$ (l)	Livello attuale del carburante.
gear	-1,1,2,3,4,5,6	Valore Marcia
rpm	$[0, +\infty)$ (giri/min)	Numero di giri al minuto del motore dell'auto
lastLapTime	$[0, +\infty)$ (s)	Tempo impiegato per completare l'ultimo giro
opponents	$[0, 200]$ (m)	Vettore di 36 sensori di avversari: ogni sensore copre un arco di 10 gradi entro un raggio di 200 metri e restituisce la distanza dell'avversario più vicino nell'area coperta. I 36 sensori coprono tutto lo spazio intorno all'auto, spaziando in senso orario da -180° fino a $+180^\circ$ gradi rispetto all'asse della macchina.

API TORCS / INPUT DEL NOSTRO ALGORITMO

Spazio degli stati principali

Nome Sensore	Range	Descrizione
racePos	{1,2,- - -,N}	Posizione in gara rispetto alle altre auto.
speedX	$(-\infty, +\infty)$	Velocità dell'auto lungo l'asse longitudinale dell'auto.
speedY	$(-\infty, +\infty)$	Velocità dell'auto lungo l'asse trasversale dell'auto.
speedZ	$(-\infty, +\infty)$	Velocità dell'auto lungo l'asse Z dell'auto.
wheelSpinVel	$[0, +\infty]$ (rad/s)	Vettore di 4 sensori che rappresentano la velocità di rotazione delle ruote.
z	$[-\infty, +\infty]$ (m)	Distanza del centro di massa dell'auto dalla superficie della pista lungo l'asse Z.

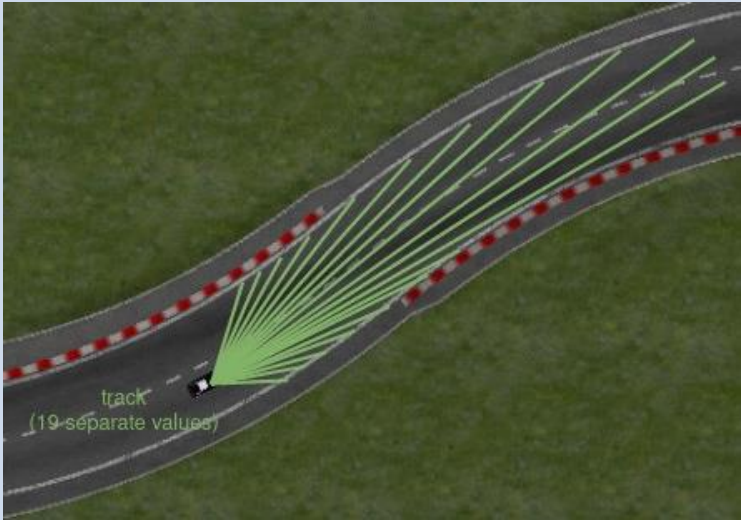
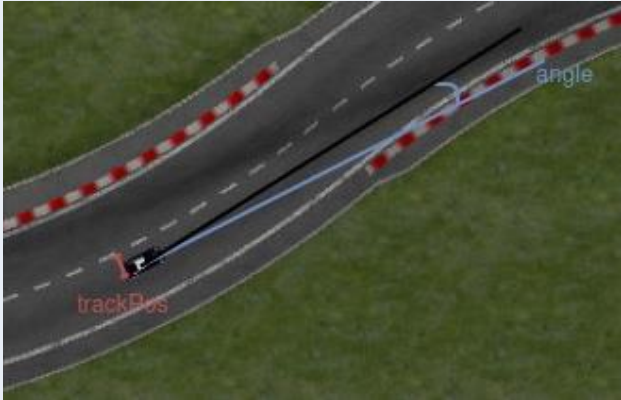
API TORCS / INPUT DEL NOSTRO ALGORITMO

Spazio degli stati principali

Nome Sensore	Range	Descrizione
track	[0, 200]	Vettore di 19 valori, uno per ciascun sensore: ogni sensore restituisce la distanza tra il bordo della pista e l'auto entro un raggio di 200 metri. Per impostazione predefinita, i sensori campionano lo spazio antistante l'auto ogni 10 gradi, con un intervallo in senso orario da -90 gradi fino a +90 gradi rispetto all'asse dell'auto. Quando l'auto è al di fuori della pista (cioè, pos è minore di -1 o maggiore di 1), i valori restituiti non sono affidabili (in genere viene restituito -1).
trackPos	$(-\infty, +\infty)$	Distanza tra l'auto e l'asse della pista. Il valore è normalizzato rispetto alla larghezza della pista: • 0 quando l'auto è sull'asse, • -1 quando l'auto è sul bordo destro della pista • +1 quando è sul bordo sinistro dell'auto. • Valori maggiori di 1 o minori di -1 significano che l'auto è fuori dalla pista.

API TORCS / INPUT DEL NOSTRO ALGORITMO

Spazio degli stati principali

Nome Sensore	Range	Descrizione
track	[0, 200]	 track (19 separate values)
trackPos	$(-\infty, +\infty)$	 trackPos angle

API TORCS / OUTPUT DEL NOSTRO ALGORITMO

Spazio delle azioni

Nome Sensore	Range	Descrizione
accel	[0, 1]	Pedale del gas virtuale (0 significa senza gas, 1 gas pieno).
brake	[0, 1]	Pedale del freno virtuale (0 significa nessun freno, 1 freno completo).
clutch	[0, 1]	Pedale virtuale della frizione (0 significa nessuna frizione, 1 frizione completa).
gear	-1,1,2,3,4,5,6	Valore Marcia
steering	[0, 1]	Valore di sterzata: -1 e +1 significano rispettivamente piena destra e sinistra, che corrispondono a un angolo di 0,366519 rad (circa 21 gradi)
meta	[0, 1]	Questo è un comando di metacontrollo: - 0 non fa nulla - 1 chiede al server di gara di riavviare la gara

LE CLASSI Python

SnakeOil – Client Python

SnakeOil è una libreria Python pensata per interfacciarsi con **TORCS**, modificato con estensioni server che permettono di controllare l'auto dall'esterno.

La **SnakeOil interface** è stata sviluppata in occasione della **Simulated Car Racing Championships del 2013**. Essa permette di:

- scrivere rapidamente un **controller per un'auto autonoma** in TORCS;
- evitare tutta la parte complicata di configurazione del simulatore;
- concentrarsi **solo sulla logica di guida** (sterzo, accelerazione, frenata, gestione dei sensori);
- lavorare in **Python**, senza dover affrontare linguaggi o dettagli tecnici più scomodi.

SnakeOil si occupa della comunicazione con TORCS, così voi potete dedicarvi alla parte davvero interessante: **far guidare l'auto nel modo più intelligente ed efficace possibile**.



Dizionari Stato ed Azioni

Il client contiene due oggetti fondamentali:

- **ServerState (C.S)** → contiene tutti i dati inviati dal server (sensori, stato dell'auto, informazioni di gara).
- **DriverAction (C.R)** → contiene i comandi che il tuo bot invia al simulatore (acceleratore, sterzo, freno, ecc.).

Entrambi hanno un dizionario interno `d` che raccoglie i valori con la sintassi del server. Questo approccio permette di adattarsi facilmente a eventuali modifiche future del protocollo.

Per leggere un valore di stato: `X= C.S.d['tracPos']`

Per impostare una azione: `C.R.d['steer']= X`

SnakeOil – Utilizzo

Per usare SnakeOil è sufficiente **importare la libreria** e definire una funzione **drive()**. SnakeOil si occuperà automaticamente della gestione delle opzioni, della connessione al server TORCS e di tutta la comunicazione necessaria.

Qui sotto trovi un esempio completo di **client funzionante**, utile come base per scrivere il tuo controller personalizzato.

```
#!/usr/bin/python
import snakeoil

if __name__ == "__main__":
    C = snakeoil.Client()
    for step in xrange(C.maxSteps, 0, -1):
        C.get_servers_input()
        snakeoil.drive_example(C)
        C.respond_to_server()
    C.shutdown()
```

- Crea un client (C = snakeoil.Client())
- Per ogni step della simulazione:
 - Riceve i dati dai sensori dell'auto (get_servers_input())
 - Esegue una funzione di guida di esempio (drive_example())
 - Invia i comandi al simulatore (respond_to_server())
- Alla fine chiude la connessione (shutdown())

Script torcs_jm_par_modulare.py

Il file è una libreria per interagire con un simulatore di guida (TORCS). È organizzato principalmente attorno a **tre classi fondamentali**, più alcune funzioni di supporto.

Classe Client

È la classe centrale che gestisce:

- la connessione al server di simulazione
- lo scambio di dati (input/output)
- il ciclo di simulazione

Classe ServerState

Rappresenta lo **stato attuale della macchina** nel simulatore.

Contiene:

- tutti i dati provenienti dal server (velocità, angolo, posizione, ecc.)
- una struttura dati (d) che memorizza questi valori

Classe DriverAction

Rappresenta le **azioni del pilota** nel simulatore.

Contiene accelerazione, frenata, sterzata, marcia, ecc.

Serve per costruire la risposta da inviare al server.

Script torcs_jm_par_modulare.py

Il file è una libreria per interagire con un simulatore di guida (TORCS). Oltre alle tre classi fondamentali, include **alcune funzioni esterne** di supporto.

`clip()` → limita un valore tra minimo e massimo

`bargraph()` → crea grafici ASCII per Debug

`destringify()` → converte stringhe in numeri

`drive_example()` → esempio di programma per guidare il veicolo

Metodi della classe Client

__init__()

Inizializza parametri (host, porta, ecc.)

Crea:

- ServerState (S)
- DriverAction (R)

Avvia la connessione

setup_connection()

Crea socket UDP

Stabilisce connessione con il server

Invia un messaggio iniziale

parse_the_command_line()

Legge parametri da riga di comando

Permette configurazione del programma

get_servers_input()

Riceve dati dal server

Li passa a ServerState

respond_to_server()

Invia al server le azioni del pilota (DriverAction)

shutdown()

Chiude la connessione

Metodi della classe ServerState

`__init__()`

Inizializza:

- La variabile che conterrà la stringa che si riceverà dal server
- dizionario dati (d)

`parse_server_str(server_string)`

Converte la stringa ricevuta dal server in dati strutturati. Usa `destringify()`.

`__repr__()`

Rappresentazione testuale dello stato

`fancyout()`

Utile per il debug. Genera grafici ASCII per rappresentare velocità, posizione e sensori da mostrare sulla console.

Metodi della classe DriverAction

`__init__()`

Definisce i valori iniziali delle azioni intraprese dall'agente

`clip_to_limit()`

Limita i valori delle singole azioni nei range ammessi

`_repr_()`

Converte le azioni in una stringa da inviare al server

`fancyout()`

Utile per il debug. Genera grafici ASCII delle azioni.

Drive_modular(c)

```
def drive_modular(c):  
    S, R = c.S.d, c.R.d  
    R['steer'] = calculate_steering(S)  
    R['accel'] = calculate_throttle(S, R)  
    R['brake'] = apply_brakes(S)  
    R['accel'] = traction_control(S, R['accel'])  
    R['gear'] = shift_gears(S)  
    return
```

S = **ServerState** → dizionario con sensori

R = **DriverAction** → dizionario dei comandi

Sequenza:

1. calcola sterzo
2. calcola accelerazione
3. applica freno
4. applica controllo trazione
5. cambia marcia

Moduli Drive_modular(c)

```
import mathD
```

```
# ===== USER CONFIGURABLE PARAMETERS =====
```

```
TARGET_SPEED = 100 # Target speed in km/h. Increasing this makes the car go faster but may reduce stability.
STEER_GAIN = 30 # Steering sensitivity. Higher values make the car turn more aggressively.
CENTERING_GAIN = 0.20 # How strongly the car corrects its position toward the center of the track.
BRAKE_THRESHOLD = 0.9 # Angle threshold for braking. Lower values brake earlier.
GEAR_SPEEDS = [0, 20, 40, 80, 100, 180] # Speed thresholds for gear shifting.
ENABLE_TRACTION_CONTROL = True # Toggle traction control system.
```

```
# ===== HELPER FUNCTIONS =====
```

```
def calculate_steering(S):
    steer = (S['angle'] * STEER_GAIN / math.pi) - (S['trackPos'] * CENTERING_GAIN)
    return max(-1, min(1, steer))
```

```
def calculate_throttle(S, R):
    if S['speedX'] < TARGET_SPEED - (R['steer'] * 2.5):
        accel = min(1.0, R['accel'] + 0.4)
    else:
        accel = max(0.0, R['accel'] - 0.2)
    if S['speedX'] < 10:
        accel += 1 / (S['speedX'] + 0.1)
    return max(0.0, min(1.0, accel))
```

```
def apply_brakes(S):
    return 0.3 if abs(S['angle']) > BRAKE_THRESHOLD else 0.0
```

```
def shift_gears(S):
    gear = 1
    for i, speed in enumerate(GEAR_SPEEDS):
        if S['speedX'] > speed:
            gear = i + 1
    return min(gear, 6)
```

```
def traction_control(S, accel):
    if ENABLE_TRACTION_CONTROL:
        if ((S['wheelSpinVel'][2] + S['wheelSpinVel'][3]) - (S['wheelSpinVel'][0] + S['wheelSpinVel'][1])) > 2:
            accel -= 0.1
    return max(0.0, accel)
```

Combina angolo rispetto alla pista e posizione laterale per mantenere l'auto centrata ed effettuare le curve

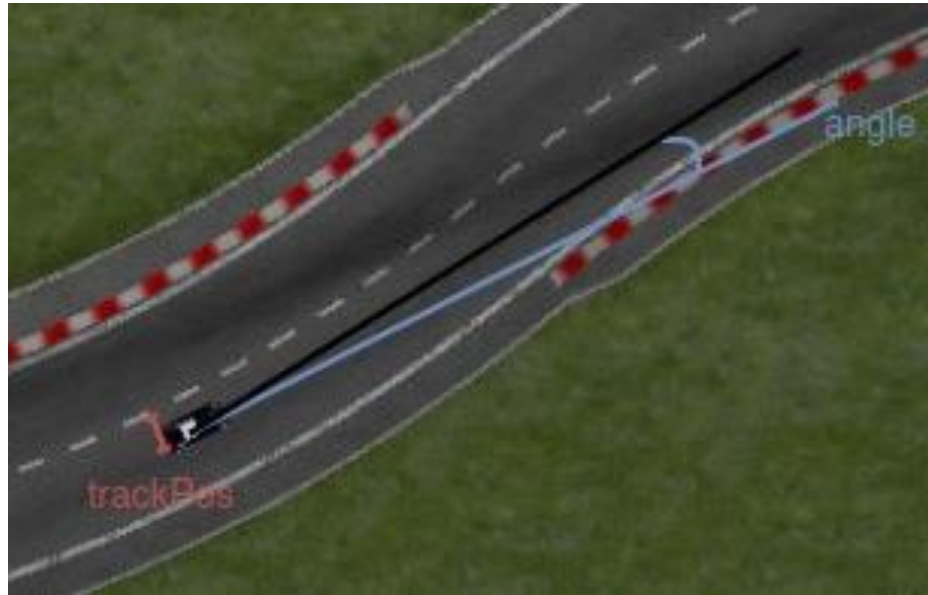
Controllo delle velocità dell'auto:
Se non abbiamo raggiunto la velocità target, acceleriamo, altrimenti rallentiamo

Frenata basata sulla curva del percorso

Cambio automatico della marcia sulla base della velocità

Controllo slittamento ruote: se slittano, riduco accelerazione

Client JAVA - SimpleDriver



```
def calculate_steering(S):  
    steer = (S['angle'] * STEER_GAIN / math.pi) - (S['trackPos'] * CENTERING_GAIN)  
    return max(-1, min(1, steer))
```

Come approcciare al problema?

- Utilizzare un approccio di behavioral cloning
- E' la più semplice forma di «*imitation learning*»
 - Il sistema intelligente apprende la «policy» utilizzando la coppia osservazione-azione



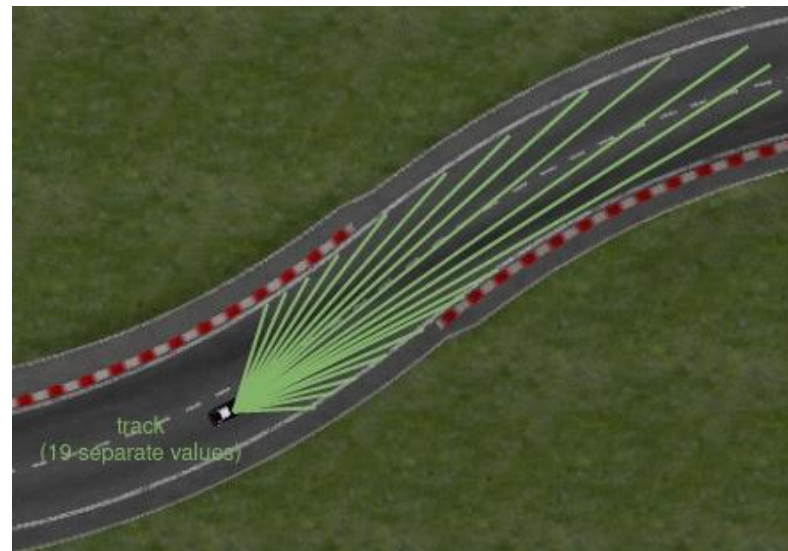
Behavioral cloning

- **INPUT / Osservazione**

- L'osservazione rappresenta «il nostro vettore di feature» (es: il vettore track)

- **OUTPUT / Azione**

- L'azione rappresenta il «comando» che, dato quell'input, l'operatore umano ha effettuato
- Il comando è espresso in termini di velocità target e sterzata (che è quello che si serve per guidare il veicolo nel nostro simulatore)



Cosa mi serve per realizzare questo sistema?

- Acquisizione del dataset
 - L' «esperto» del mio dominio colleziona le coppie osservazione-azione
 - Effettuando numerosi giri, in modo manuale, e salvando le relative coppie «dato letto dai sensori» - «azione effettuata dal pilota»
- Dato il «dataset», utilizzo uno dei classificatori studiati (es: K-NN), al fine di produrre l'output desiderato (il comando) a partire dall'input («i dati letti dai sensori»)
- Attenzione: se entro in uno stato molto distante da ciò che ho imparato, la decisione che prenderò molto probabilmente sarà sbagliata... Ma se uso troppi campioni, il mio K-NN (o NN) sarà troppo lento...

